Final Project: Enterprise-Grade RAG Workflow Application

Assignment Type: Final Project

20 Marks

1. Project Overview

The objective of this final project is to move beyond basic LLM integrations and build a robust, production-ready Application utilizing Retrieval-Augmented Generation (RAG).

You are required to build a **process-driven application** (NOT just a conversational chatbot) that processes a massive corpus of documents, maintains strict security constraints, ensures high accuracy and low latency, features advanced prompt engineering techniques, and guarantees graceful degradation in the event of API failures.

2. Core Technical Requirements

A. Massive Scale & Dynamic Data Management

- **Scale:** Your knowledge base must consist of a minimum of **5,000 pages** of unstructured or semi-structured documents (e.g., PDFs, manuals, financial reports).
- **Dynamic Indexing (CRUD):** The system must not rely on a static, one-time vector database build. You must implement functionality allowing users to **add, modify, or delete** documents or specific pages at any stage without rebuilding the entire index.
- *Hint:* Utilize page-level indexing and robust metadata tagging (document IDs, page numbers) in your vector database to enable targeted upserts/deletions.

B. Application & Process Integration (No Standard Chatbots)

- **Workflow Automation:** The application must use RAG in the background to execute a specific workflow.
- *Examples:* * An automated compliance auditor that reads user-uploaded contracts and flags violations against a 5,000-page policy corpus.
    - A research assistant that ingests a query, pulls data from the corpus, and automatically generates a structured, cited 3-page PDF report.
    - A customer support routing system that reads an incoming ticket, finds the solution in the docs, and automatically drafts an email and sets ticket priority.

C. Advanced Retrieval for Accuracy & Latency

- **Handling Irrelevant Chunks:** RAG systems often retrieve noisy, irrelevant data which hurts both accuracy and latency. You must implement strategies to filter this out.

- **Requirement:** Implement an advanced retrieval pipeline (e.g., "Retrieve and Rerank"). Use a fast vector search for initial retrieval (recall) and a cross-encoder/reranker (like Cohere Rerank or FlashRank) for precision filtering before sending chunks to the LLM.

D. Advanced Prompting & Token Optimization

- **System Prompts:** You must utilize well-crafted System Prompts to establish the AI's persona, strict behavioral boundaries, and output formatting rules.
- **Reasoning Frameworks:** Depending on your application's use case, you must integrate advanced prompting methods such as **Chain of Thought (CoT)**, **Tree of Thoughts (ToT)**, or **ReAct** (Reasoning and Acting) to handle complex logical workflows or multi-step analysis.
- **Token Efficiency:** Everything must be optimized for minimal token usage. This includes writing concise prompts, dynamically summarizing or stripping useless characters/metadata from retrieved chunks before they hit the context window, and utilizing max-token limiters effectively.

E. Security & Guardrails

- **Input Guardrails:** Implement checks to block prompt injections, jailbreak attempts, or out-of-scope queries before they reach the LLM.
- **Output Guardrails:** Ensure the system checks the LLM's output for hallucinations against the retrieved context, blocking unverified claims.

F. Resiliency & API Failover

- **Graceful Degradation:** External LLM APIs (OpenAI, Anthropic, Gemini, etc.) fail, timeout, or hit rate limits.
- **Requirement:** If the primary LLM API fails, the application must not crash. It must catch the error and execute a fallback mechanism. The fallback must provide a partial RAG response (e.g., gracefully displaying the exact retrieved raw text chunks/pages to the user with a warning message that AI generation is offline).

3. Development Guidelines: Open Source & "Vibe Coding"

You are highly encouraged to reference open-source projects and utilize AI-assisted coding tools (e.g., "vibe coding" with Cursor, Copilot, or ChatGPT) to accelerate your development process.

However, there is a strict boundary:

- **No Blind Copying:** You must **not** clone an entire repository or blindly copy-paste an entire project and label it as your own work.
- **Modification & Customization:** You are required to actively modify, customize, and stitch the architecture together for your specific use case.
- **Learn the Concepts:** This project covers highly sought-after, enterprise-critical concepts. This is the absolute best time for you to genuinely learn these mechanisms by getting your

hands dirty. Be prepared to explain and defend every line of code, prompt, and architectural decision in your submission.

4. Deliverables

1. **Source Code:** Complete application repository (GitHub link).
2. **Architecture Diagram:** A visual map of your RAG pipeline, indexing strategy, prompt workflows (e.g., CoT loops), guardrails, and fallback mechanisms.
3. **Application Demo:** A live demo or recorded video (max 5 minutes) demonstrating:
    - The primary workflow.
    - Adding/modifying a document dynamically.
    - Triggering a guardrail (input or output).
    - Simulating an API failure to show the partial RAG fallback.
4. **Project Report (10 pages):** Detailing your embedding choices, chunking strategy, vector database, reranking method, prompt engineering strategies (CoT/ToT), token/latency optimization techniques, and any open-source libraries used.

5. Grading Rubric

Category	Description	Weight
System Architecture & Scale	Successfully handles 5000+ pages; dynamic CRUD capabilities function correctly using metadata.	20%
Retrieval Quality & Latency	Implements reranking/advanced retrieval; minimizes irrelevant chunks sent to context; latency is optimized.	20%
Application Process	RAG is integrated into a tangible workflow/utility, going beyond a simple conversational chatbot UI.	20%
Prompting & Token Optimization	Implements robust system prompts, appropriate reasoning strategies (CoT/ToT), and strict token management.	15%
Security & Guardrails	Effectively blocks malicious/irrelevant inputs and prevents hallucinated outputs.	15%
Resiliency (Fallback)	System successfully intercepts API failures and provides a raw-chunk partial RAG answer without crashing.	10%

Good luck! Focus on systems engineering, edge cases, prompt efficiency, and building a resilient pipeline.